

Assignment 1 (Round 2) - Inter-Process Communication & Memory Isolation

Student Name: Jerome Arsany Mansour Farah

ID: 2305093

Course: Operating System Security

Part 1: Secure Inter-Process Communication (ipc_secure.c)

Objective:

Create a C program that utilizes pipe() and fork() to establish a unidirectional communication channel. The parent process sends a message, and the child process reads and validates it against a predefined security rule.

Source Code:

```
` codeC
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int pipe_fd[2];
    pid_t pid;
    char buffer[256];
    char *message = "SECRET: OS Security is passing!";

    // Create the pipe
    if (pipe(pipe_fd) == -1) {
        perror("Pipe failed");
        exit(1);
    }

    // Call fork() to create a child process
    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    if (pid > 0) {
        // --- PARENT PROCESS ---
```

```

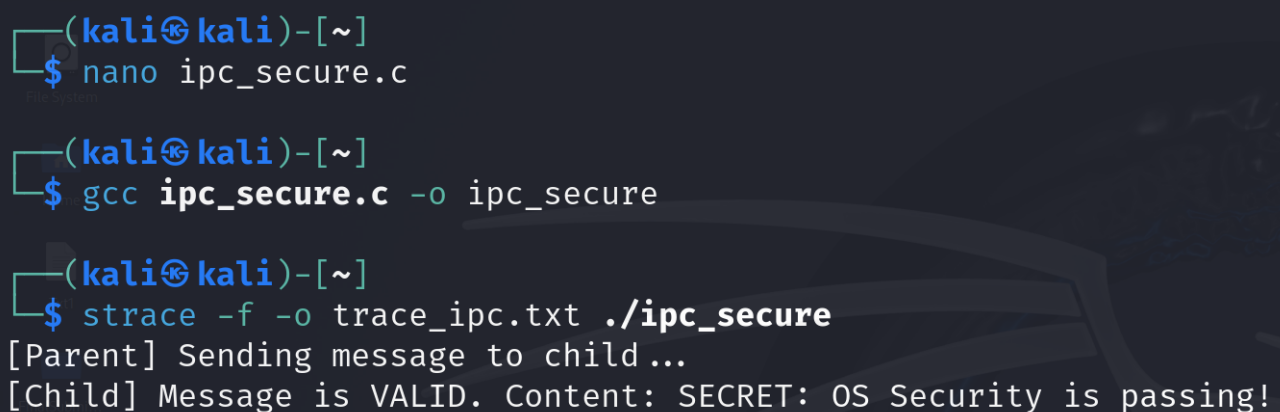
        close(pipe_fd[0]); // Close unused read end
        printf("[Parent] Sending message to child...\n");
        write(pipe_fd[1], message, strlen(message) + 1);
        close(pipe_fd[1]); // Close write end after sending
        wait(NULL); // Prevent zombie process
    } else {
        // --- CHILD PROCESS ---
        close(pipe_fd[1]); // Close unused write end
        read(pipe_fd[0], buffer, sizeof(buffer));
        close(pipe_fd[0]); // Close read end after receiving

        // Validate message (Predefined rule: must start with
"SECRET:")
        if (strncmp(buffer, "SECRET:", 7) == 0) {
            printf("[Child] Message is VALID. Content: %s\n",
buffer);
        } else {
            printf("[Child] ERROR: Invalid message format!
Rejecting.\n");
        }
        exit(0);
    }
    return 0;
} "

```

Execution Output & strace Verification:

The program was compiled and run using the command: `strace -f -o trace_ipc.txt ./ipc_secure`. The `-f` flag was explicitly used to ensure strace followed the child process created by the `fork()` system call (which maps to `clone()` in the Linux kernel).



```

(kali㉿kali)-[~]
$ nano ipc_secure.c

(kali㉿kali)-[~]
$ gcc ipc_secure.c -o ipc_secure

(kali㉿kali)-[~]
$ strace -f -o trace_ipc.txt ./ipc_secure
[Parent] Sending message to child...
[Child] Message is VALID. Content: SECRET: OS Security is passing!

```

Technical Discussion & Defense:

- **File Descriptors (pipe_fd):** The `pipe()` system call populates an array with two file descriptors: `pipe_fd[0]` for reading and `pipe_fd[1]` for writing.

- **Unidirectional Enforcement:** To prevent race conditions and ensure secure unidirectional flow, the parent explicitly closes its read end (`close(pipe_fd[0])`), and the child explicitly closes its write end (`close(pipe_fd[1])`).
- **Validation Rule:** The child uses `strncmp` to verify the first 7 bytes of the buffer match the keyword "SECRET:". This simulates basic input validation/sanitization to prevent processing malicious payloads.

Part 2: Process Memory Isolation (mem_isolation.c)

Objective:

Demonstrate that processes created via `fork()` are entirely isolated by the Operating System. The experiment modifies both static (stack) and dynamically allocated (heap) variables to prove that modifying memory in the child does not affect the parent.

Source Code:

```
` codeC
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    // 1. Declare a variable (Stack Memory)
    int x = 10;

    // 2. Dynamically allocate a second variable (Heap Memory)
    int *y = (int *)malloc(sizeof(int));
    *y = 20;

    // Call fork() to create a child process
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    if (pid > 0) {
        // --- PARENT PROCESS ---
        x = 50;
        *y = 60;

        printf("=== PARENT PROCESS ===\n");
        printf("Value of x: %d\n", x);
        printf("Address of x: %p\n", (void *)&x);
        printf("Value of *y (dynamic): %d\n", *y);
    }
}
```

```

        printf("Address of y (dynamic): %p\n\n", (void *)y);

        wait(NULL);
        free(y);
    } else {
        // --- CHILD PROCESS ---
        sleep(1); // Synchronization delay for clean output

        x = 100;
        *y = 200;

        printf("=== CHILD PROCESS ===\n");
        printf("Value of x: %d\n", x);
        printf("Address of x: %p\n", (void *)&x);
        printf("Value of *y (dynamic): %d\n", *y);
        printf("Address of y (dynamic): %p\n\n", (void *)y);

        free(y);
        exit(0);
    }
    return 0;
} "

```

Execution Output & strace Verification:

The program was compiled and run using the command: `strace -f -o trace_mem.txt ./mem_isolation`.

```

(kali㉿kali)-[~]
$ nano mem_isolation.c

(kali㉿kali)-[~]
$ gcc mem_isolation.c -o mem_isolation

(kali㉿kali)-[~]
$ strace -f -o trace_mem.txt ./mem_isolation
== PARENT PROCESS ==
Value of x: 50
Address of x: 0x7ffcd7995710
Value of *y (dynamic): 60
Address of y (dynamic): 0x5557066212a0

== CHILD PROCESS ==
Value of x: 100
Address of x: 0x7ffcd7995710
Value of *y (dynamic): 200
Address of y (dynamic): 0x5557066212a0

```

Technical Discussion & Defense:

- **Identical Virtual Addresses:** The execution output proves that the memory addresses for x (stack) and y (heap) are *exactly the same* in both the parent and child processes.
- **Memory Isolation & Copy-On-Write (COW):** Despite sharing the exact same address, the *values* are completely different (x=50 in parent, x=100 in child). This is because the OS assigns each process its own isolated **Virtual Address Space**. When fork() is called, the OS uses a security optimization called **Copy-On-Write**. The child is mapped to the parent's physical memory pages initially (marked as read-only). The moment the child attempts to modify x or y, the CPU triggers a page fault. The OS then secretly allocates a *new* physical frame of RAM for the child and updates the child's page table.
- **Conclusion:** The addresses printed are merely virtual illusions provided by the OS. The physical memory is strictly isolated, ensuring one process cannot corrupt the memory of another.